

PROJECT OVERVIEW

Aircraft Maintenance Data Warehouse

Turning raw airline operations data into reliable business KPIs with SQL, an ETL pipeline and a dimensional data model.

Goal	Give an airline a single, trustworthy place to measure how much its aircraft fly, how often they are grounded, how punctual they are, and how often technical issues are reported.
What I built	A data warehouse plus an automated pipeline that extracts data from two operational systems, cleans it with business rules and loads it ready for analysis.
Outcome	14 KPIs, grouped by aircraft manufacturer and year, each answered by a short SQL query instead of a long query over raw data.
Tech stack	SQL (PostgreSQL, DuckDB) · Python · pandas · pygrametl · pytest
Context	Academic team project (2 people), Data Science and Engineering degree, Universitat Politècnica de Catalunya (UPC).

In one minute

Airlines generate huge amounts of operational data, but it is spread across systems that were built to *run* the operation, not to *analyse* it. Asking a simple question such as “**How reliable were our Airbus aircraft last year?**” required complex queries that recalculated everything from raw timestamps each time, and the data contained errors that silently distorted the results.

This project solves that by building a **data warehouse**: a database designed specifically for analysis. Data is extracted from the source systems, cleaned and reshaped once, and stored in a structure where every business question becomes a short, fast SQL query.

Skills demonstrated

- **SQL**: joins, CTEs, aggregations, conditional logic, safe division, schema and constraint design.
- **Data modeling**: designing fact and dimension tables (galaxy schema) at the right level of detail.
- **Data engineering**: an Extract-Transform-Load (ETL) pipeline in Python with bulk loading.
- **Data quality**: detecting and fixing inconsistent records, with an audit log of every change.
- **Software practices**: modular code, unit tests, credentials kept out of the code.

1. The business problem

The airline's data lives in two operational systems:

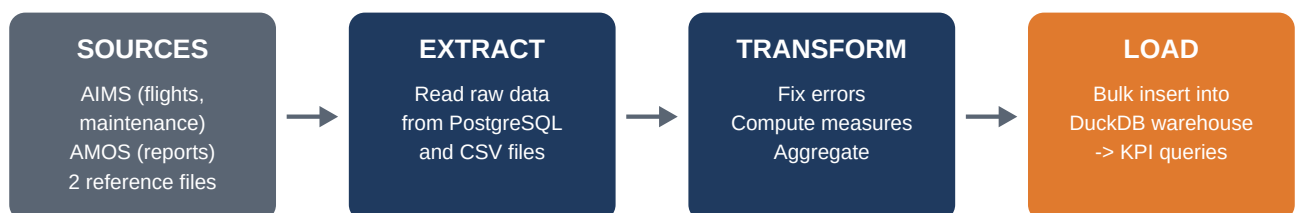
System	What it contains
AIMS	Every flight (scheduled and actual departure/arrival times, cancellations) and every maintenance slot during which an aircraft is out of service.
AMOS	Technical reports written by pilots after a flight and by maintenance staff.
Reference files	Aircraft model and manufacturer; airport where each maintenance employee works.

Management wanted to track three families of indicators, always broken down by **aircraft manufacturer** (Airbus vs. Boeing) and **year**:

Area	Example questions
Utilization & availability	How many hours does each aircraft fly? How many days is it grounded for planned vs. unplanned maintenance?
Reliability	What share of flights are delayed or cancelled? How long is the average delay?
Reporting	How many technical issues are reported per 1,000 flight hours? Do pilots or maintenance staff report more?

2. The solution: an ETL pipeline

ETL stands for **Extract, Transform, Load**: the standard way of moving data from operational systems into an analytical database.



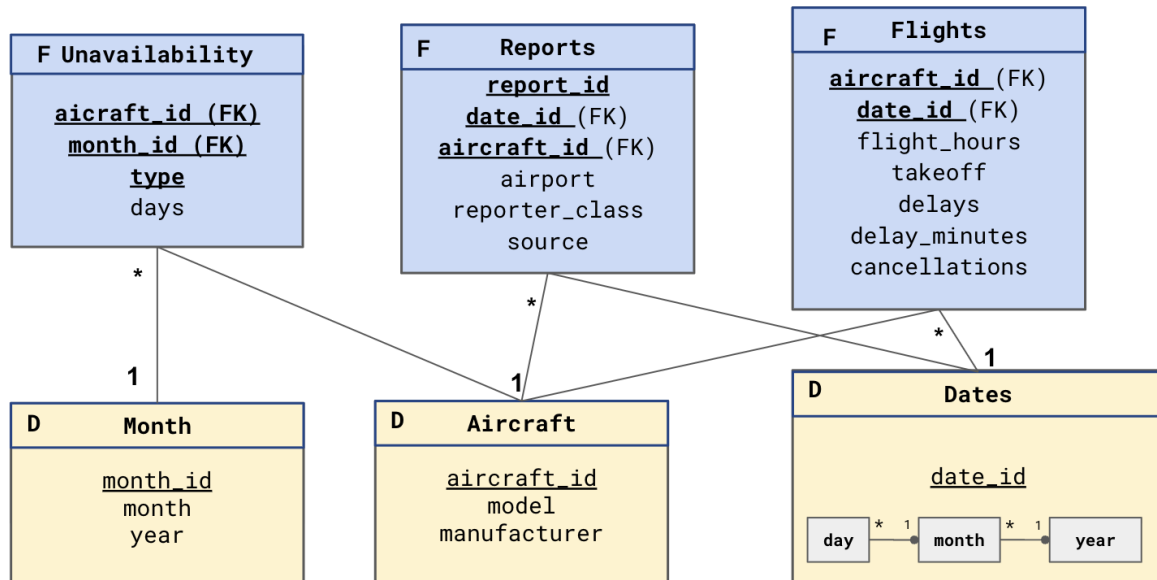
Every data correction made in TRANSFORM is written to an audit log.

Step	What happens
Extract	Python connects to the PostgreSQL source, runs SQL queries to pull flights, maintenance slots and reports, and reads the reference files.
Transform	Data-quality rules fix or discard inconsistent records. Per-flight measures (flight hours, delay minutes...) are calculated, and data is aggregated to the level each table needs.
Load	Clean data is inserted into DuckDB in bulk. Loading row by row took around 20 minutes; bulk loading makes it a matter of seconds.

3. The data model

The warehouse follows a **dimensional model**, the industry-standard design for analytics. It separates two kinds of tables:

- **Fact tables** (blue) store the events and numbers to analyse: flights, days out of service, reports.
- **Dimension tables** (yellow) store the context used to slice those numbers: which aircraft, which manufacturer, which day, month or year.



Galaxy schema: three fact tables sharing the same dimensions.

Fact table	One row per...	Main measures
Flights	aircraft and day	flight hours, take-offs, delays, delay minutes, cancellations
Unavailability	aircraft, month and maintenance type	days out of service (scheduled / unscheduled)
Reports	individual report	who reported (pilot or maintenance), airport, source system

Key design decision: two calendars

Flight indicators are defined per **day**, but availability indicators are defined per **month**. Using only a daily calendar would have meant storing 30+ almost identical rows per aircraft and month for maintenance; using only a monthly one would have made daily flight analysis impossible. Two time dimensions keep the warehouse small *and* flexible.

4. Data quality

Real operational data is messy. Three rules are applied automatically, and every correction is written to a log so that any number in a report can be traced back to its source.

Rule	Problem found in the data	How it is handled
Swapped times	Flights whose arrival time is earlier than their departure time.	Swap the two times before computing flight hours, so no flight has negative duration.
Overlapping flights	The same aircraft appearing on two flights at the same time.	Discard the earlier flight and log it.
Unknown aircraft	Reports about aircraft that are not part of the fleet.	Discard the report and log it.

Each rule is covered by an automated unit test, so the logic can be verified without access to the original database.

5. The indicators (KPIs)

All indicators are calculated per aircraft manufacturer and year.

KPI	Plain-language meaning
FH / TO	Average flight hours and take-offs per aircraft.
ADOSS / ADOSU / ADOS	Average days out of service for scheduled, unscheduled and total maintenance.
ADIS	Average days in service (365 minus days out of service).
DU / DC	Daily utilization: hours flown and take-offs per day in service.
DYR / CNR	Percentage of flights delayed by more than 15 minutes / cancelled.
TDR	Technical dispatch reliability: percentage of flights neither delayed nor cancelled.
ADD	Average delay duration of delayed flights.
RRh / RRc	Technical reports per 1,000 flight hours / per 100 flights, overall and by role (pilot vs. maintenance).

6. SQL example

Report rate per manufacturer and year. Two **CTEs** (named sub-queries) summarise flights and reports separately, and a final query joins them and divides safely (NULLIF avoids division by zero).

```
WITH flight_summary AS (  
    SELECT a.manufacturer, EXTRACT(year FROM f.date_id) AS year,  
           SUM(f.flight_hours) AS flight_hours, SUM(f.takeoff) AS flight_cycles  
    FROM Flights f JOIN Aircraft a ON f.aircraft_id = a.aircraft_id  
    GROUP BY a.manufacturer, year  
)  
,  
report_summary AS (  
    SELECT a.manufacturer, EXTRACT(year FROM r.date_id) AS year,  
           COUNT(*) AS report_count  
    FROM Reports r JOIN Aircraft a ON r.aircraft_id = a.aircraft_id  
    WHERE r.source = 'POSTFLIGHTREPORT'  
    GROUP BY a.manufacturer, year  
)  
SELECT r.manufacturer, r.year,  
       1000 * ROUND(r.report_count / NULLIF(f.flight_hours, 0), 3) AS reports_per_1000_fh,  
       100 * ROUND(r.report_count / NULLIF(f.flight_cycles, 0), 2) AS reports_per_100_flights  
FROM report_summary r  
JOIN flight_summary f USING (manufacturer, year)  
ORDER BY r.manufacturer, r.year;
```

Without the warehouse, the same answer requires reading every raw flight and report, recomputing flight durations from timestamps and mapping each aircraft to its manufacturer on the fly. The project includes a script that runs both versions side by side to compare them.

7. Engineering decisions

Decision	Why
DuckDB as the warehouse	A columnar analytical database: very fast for aggregations, runs inside Python with no server to manage.
Bulk loading	Inserting whole tables at once instead of row by row cut loading time from about 20 minutes to seconds.
Measures computed once	Flight hours and delays are calculated during the ETL, so queries never touch raw timestamps again.
Audit log	Every discarded or corrected record is logged, making results explainable.
Unit tests	Data-quality rules are tested automatically on small synthetic examples.
No secrets in code	Database credentials come from a local file or environment variables that are never committed.

8. Glossary

Term	Meaning
Data warehouse	A database designed for analysis and reporting rather than day-to-day operations.
ETL	Extract, Transform, Load: the process of moving and cleaning data into a warehouse.
Fact table	Table of measurable events (e.g. flights), with numeric values to aggregate.
Dimension table	Table of descriptive context (e.g. aircraft, date) used to group and filter facts.
Galaxy schema	A dimensional model with several fact tables that share dimensions.
CTE	Common Table Expression: a named sub-query that makes long SQL easier to read.
KPI	Key Performance Indicator: a number used to track how the business is performing.

Code and documentation are available in the GitHub repository. The original source data belongs to a private academic dataset and is not published.